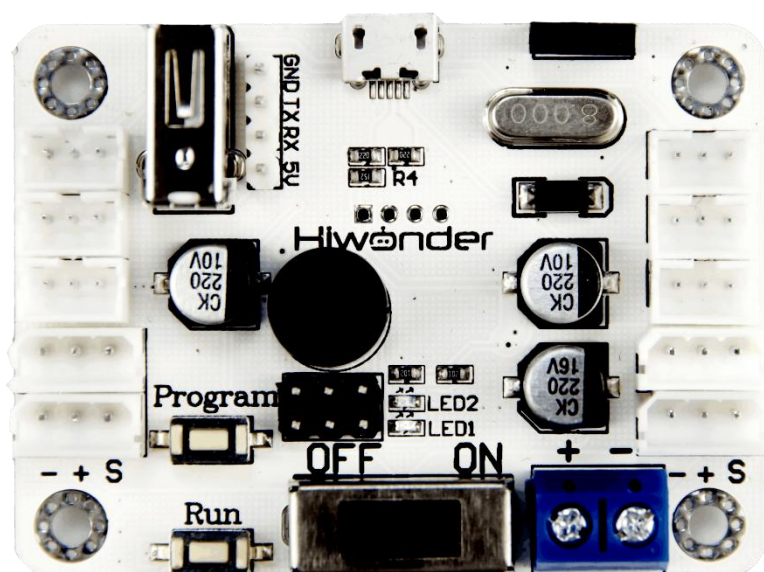


# 幻尔科技

## STM32 版本开发教程

V1.0



Hiwonder 官方网站



| 版本号  | 修改日期     | 修改摘要 |
|------|----------|------|
| V1.0 | 20231101 | 初次发布 |

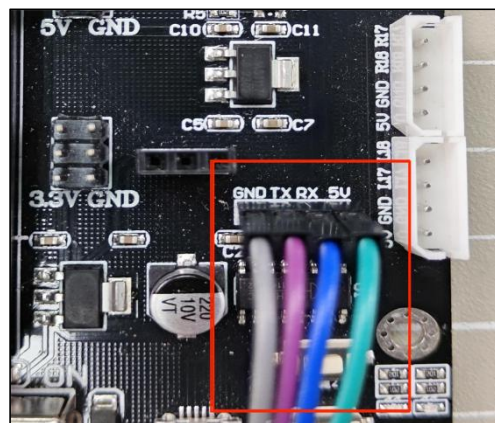
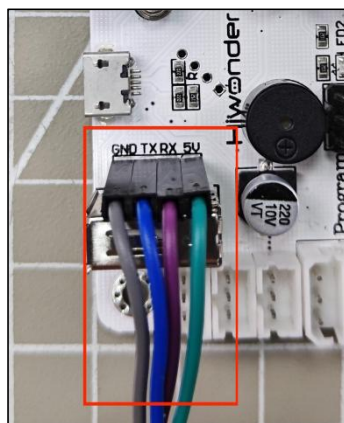
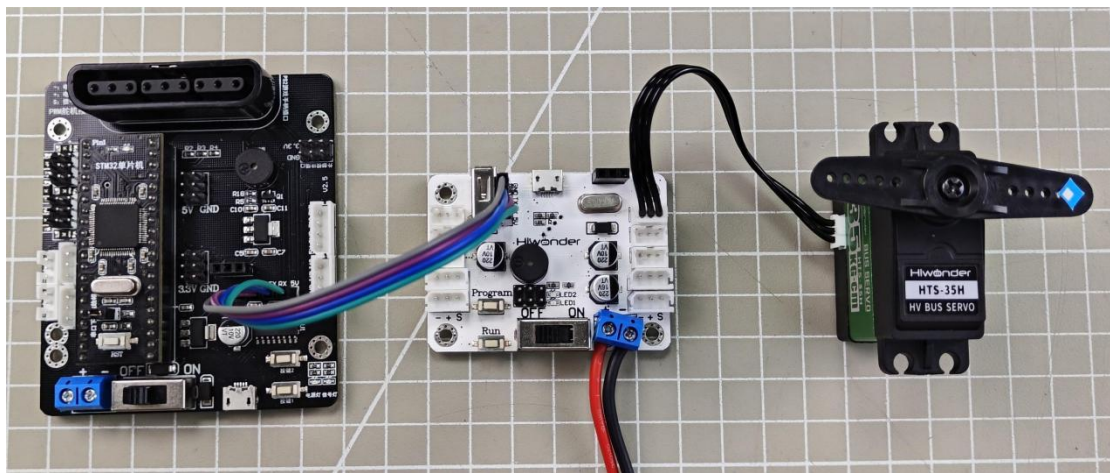
# 目录

|                          |    |
|--------------------------|----|
| 1.准备工作 .....             | 3  |
| 1.1 接线说明 .....           | 3  |
| 1.2 环境配置 .....           | 4  |
| 2.案例开发 .....             | 4  |
| 2.1 案例 1 控制单个舵机转动 .....  | 4  |
| 2.1.1 运行程序 .....         | 4  |
| 2.1.2 实现效果 .....         | 6  |
| 2.1.3 案例程序简要分析 .....     | 6  |
| 2.2 案例 2 控制单个舵机速度 .....  | 7  |
| 2.2.1 运行程序 .....         | 8  |
| 2.2.2 实现效果 .....         | 10 |
| 2.2.3 案例程序简要分析 .....     | 10 |
| 2.3 案例 3 控制多个舵机转动 .....  | 12 |
| 2.3.1 运行程序 .....         | 12 |
| 2.3.2 实现效果 .....         | 14 |
| 2.3.3 案例程序简要分析 .....     | 14 |
| 2.4 案例 4 舵机中位及偏差调节 ..... | 16 |
| 2.4.1 运行程序 .....         | 16 |
| 2.4.2 实现效果 .....         | 18 |
| 2.4.3 案例程序简要分析 .....     | 18 |

## 1. 准备工作

### 1.1 接线说明

本节示例使用的是开源舵机控制板和 STC15W4K32S4 单片机、总线控制器，通过 7.4V 6000mAh（高压舵机采用 11.1V）锂电池来供电。将总线控制器的串口 连接至开源舵机控制板的串口上。



注意：

- ① 如使用我司锂电池，连接锂电池对接线请以红接+，黑接-接到 DC 接口。
- ② 如对接线未连接锂电池，请勿与电池对接线直接对接，避免正负极发生短路而造成短路。

- ③ 连接串口时，RX 与 TX 要反接。如上图中的两个小图。

## 1.2 环境配置

在电脑端安装 Keil5，软件包位于“2.软件工具->STM32 软件工具”下。关于 Keil5 的详细使用，可在对应目录下进行学习。

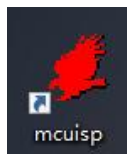
## 2.案例开发

### 2.1 案例 1 控制单个舵机转动

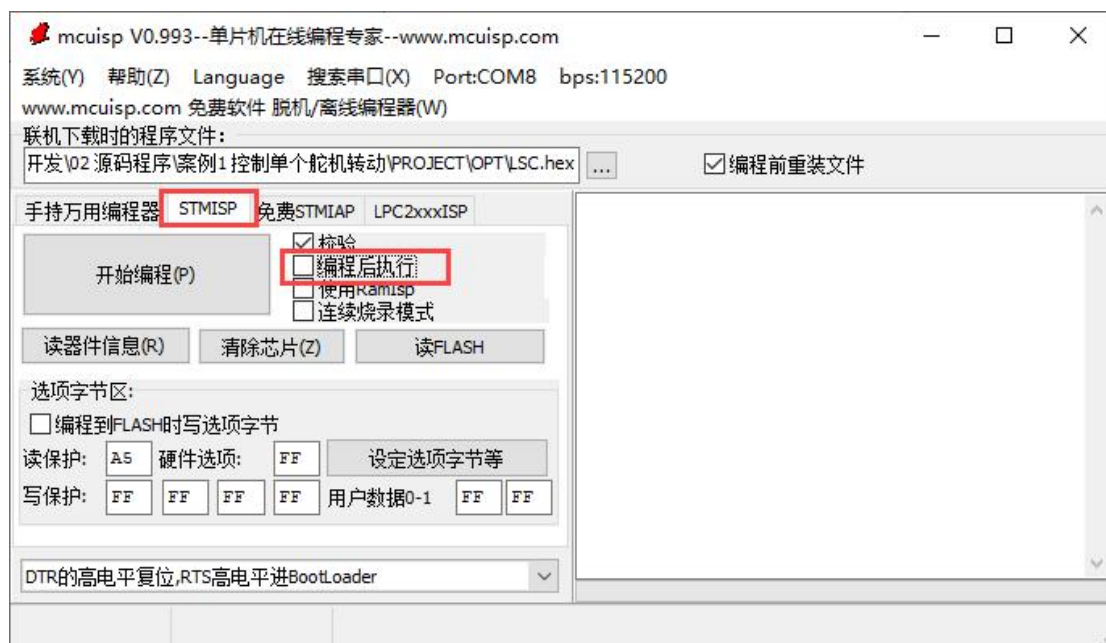
本案例串口向总线控制板发送指令，再通过总线控制板控制舵机转动。

#### 2.1.1 运行程序

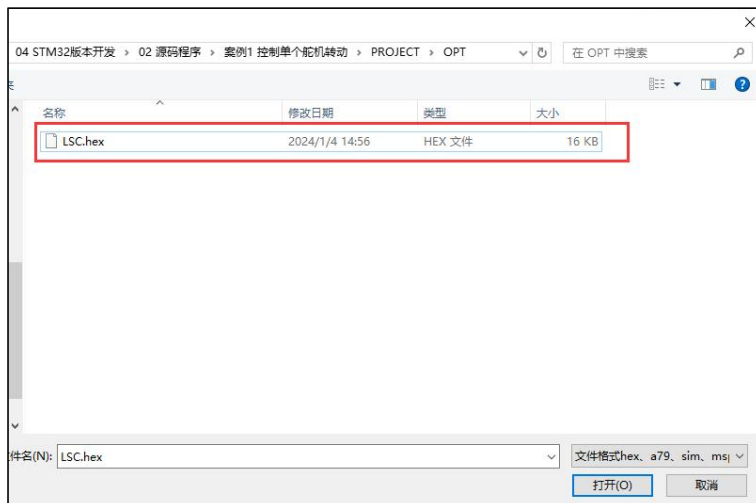
在“2.软件工具->STM32 软件工具”中打开 mcuisp 下载工具：



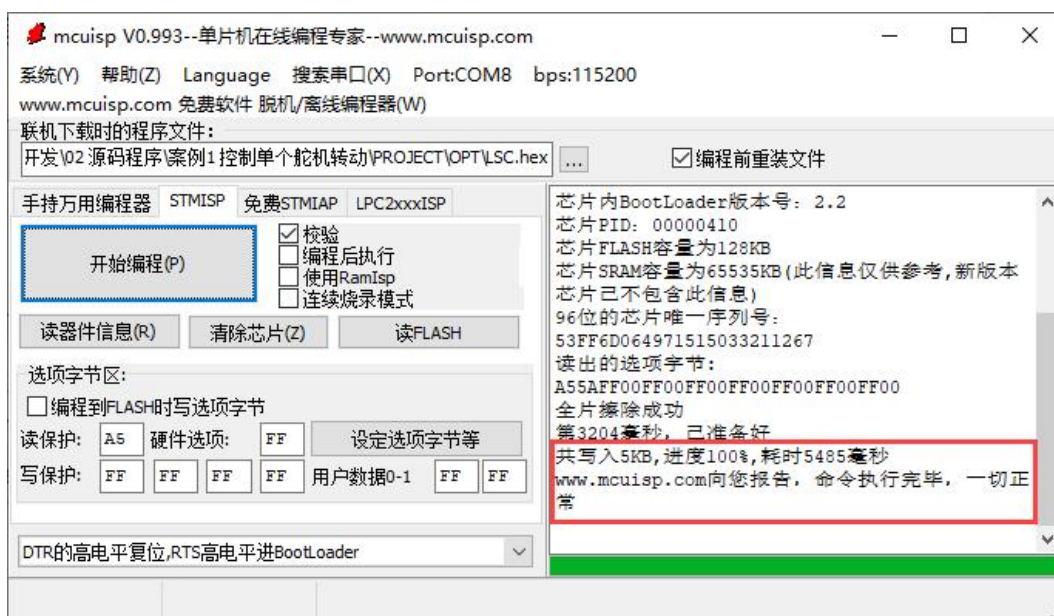
mcuisp 配置如下图：



点击“”，在“总线舵机控制器二次开发教程\04 STM32 版本开发\02 源码程序\案例 1 控制单个舵机转动\PROJECT\OPT”文件夹下找到 hex 文件：



打开程序文件后，点击“开始编程”即可将程序下载到 STM32 单片机上（注意：在下载程序时，需要先将 STM32 单片机上的跳线帽拔掉并按下复位按键，当 mcuisp 右下角显示“命令执行完毕，一切正常”则说明下载完成，最后将跳线帽插回 STM32 单片机。）



下载完成后，打开总线舵机控制器电源开关，按下 STM32 单片机的复位按键，1 号舵机开始转动。



### 2.1.2 实现效果

程序运行后，舵机会从 0 的位置到 1000 的位置之间，来回转动，间隔 1.5 秒。

### 2.1.3 案例程序简要分析

- 导入必要功能包

在 main.c 程序中导入包：

```
#include "delay.h"
#include "uart.h"
#include "LobotServoController.h"
#include "bool.h"
```

功能包主要封装用于总线控制板通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机。

```
void delay_s(int s)
{
    for(;s>0;s--)
        delay_ms(1000);
}
```

在 main.c 文件中，有一个延时函数，该函数用于在单位为：秒（s）的延时。

- 串口初始化

```
void uartInit(u32 bound)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    USART_InitTypeDef USART_InitStructure;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1 | RCC_APB2Periph_GPIOA | RCC_APB2Periph_AFIO, ENABLE);

    //USART1_TX PA9
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART1_RX PA10
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART 配置
    USART_InitStructure.USART_BaudRate = bound; //波特率
    USART_InitStructure.USART_WordLength = USART_WordLength_8b; //数据位
    USART_InitStructure.USART_StopBits = USART_StopBits_1; //停止位
    USART_InitStructure.USART_Parity = USART_Parity_No; //校验位
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None; //流控制
    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //发送接收

    USART_Init(USART1, &USART_InitStructure);
    uartNVICInit();
    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
    USART_Cmd(USART1, ENABLE);
}
```

在 `uart.c` 文件下，初始化串口，配置 PA9 为 TX 输出数据，PA10 为 RX 读取数据。配置 USART 为 8 为数据位，1 位停止位，无校验位，无流控制，可读可写模式。

### ● 控制舵机转动

```
int main(void)
{
    SystemInit(); //系统时钟等初始化
    delay_init(72); //延时初始化
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //设置NVIC中断分组2:2位抢占优先级，2位响应优先级
    uartInit(9600); //串口初始化为9600

    while(1) {
        moveServo(1, 1000, 1200); //1号舵机至1000位置
        delay_s(2);
        moveServo(1, 0, 1200); //1号舵机至0位置
        delay_s(2);
    }
}
```

开始主程序运行前，调用 `SystemInit()` 函数初始化系统时钟；调用 `delay_init(72)` 初始化延时程序；初始化 NVIC 中断控制器，它的作用是通过系统时钟产生中断来提供延时；初始化串口波特率为 9600。

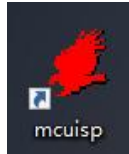
在 `while` 循环中，调用 `moveServo` 函数向总线舵机控制器发送数据，控制舵机转动。

## 2.2 案例 2 控制单个舵机速度

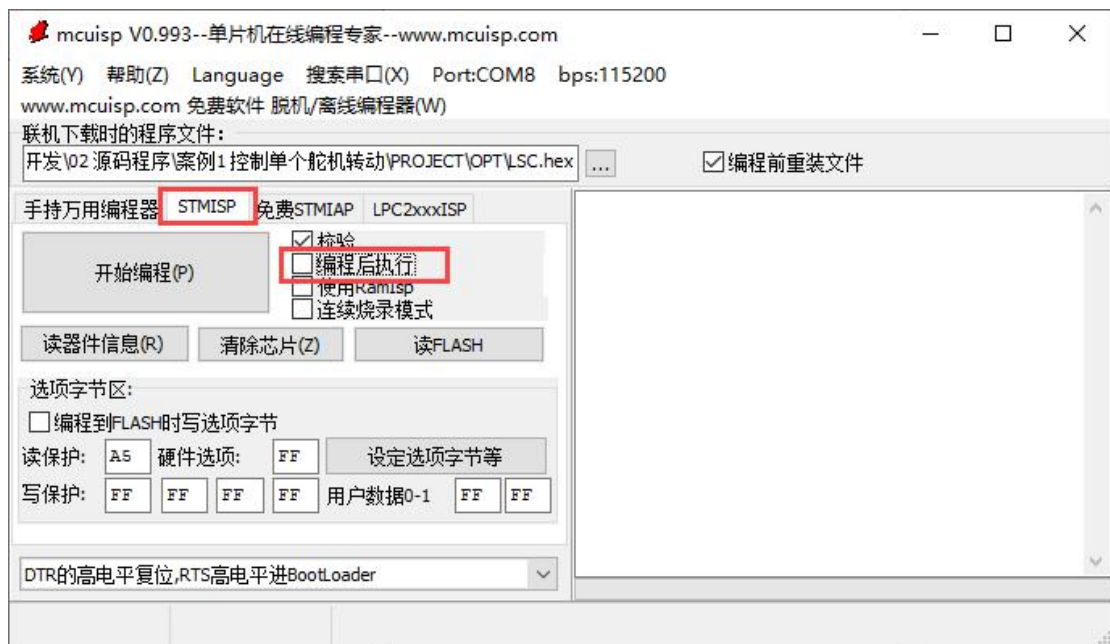
本案例串口向总线控制板发送指令，再通过总线控制板控制舵机转动。

### 2.2.1 运行程序

在“2.软件工具->STM32 软件工具”中打开 mcuisp 下载工具：

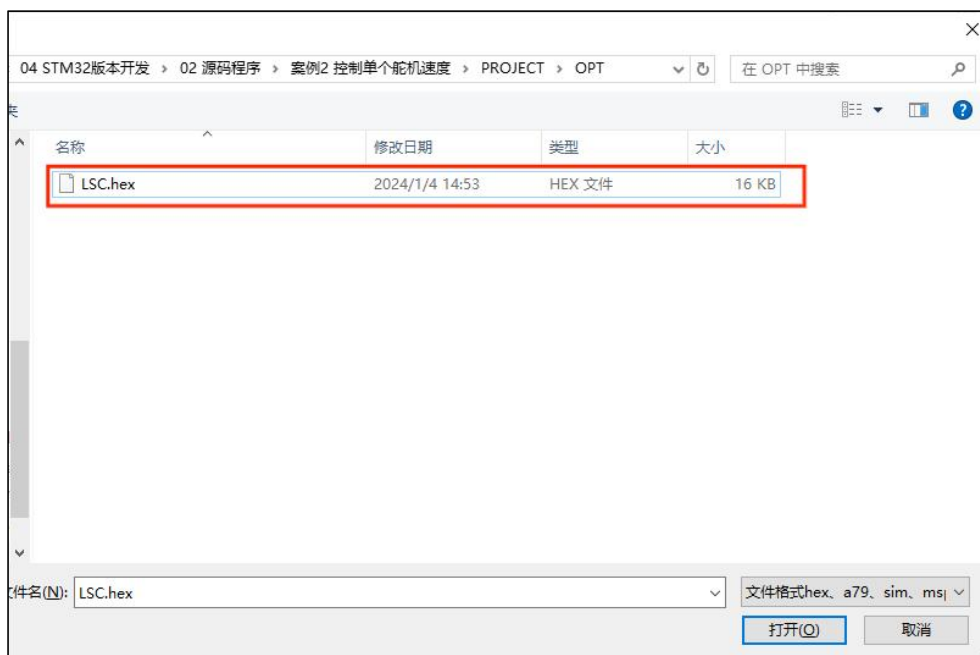


mcuisp 配置如下图：

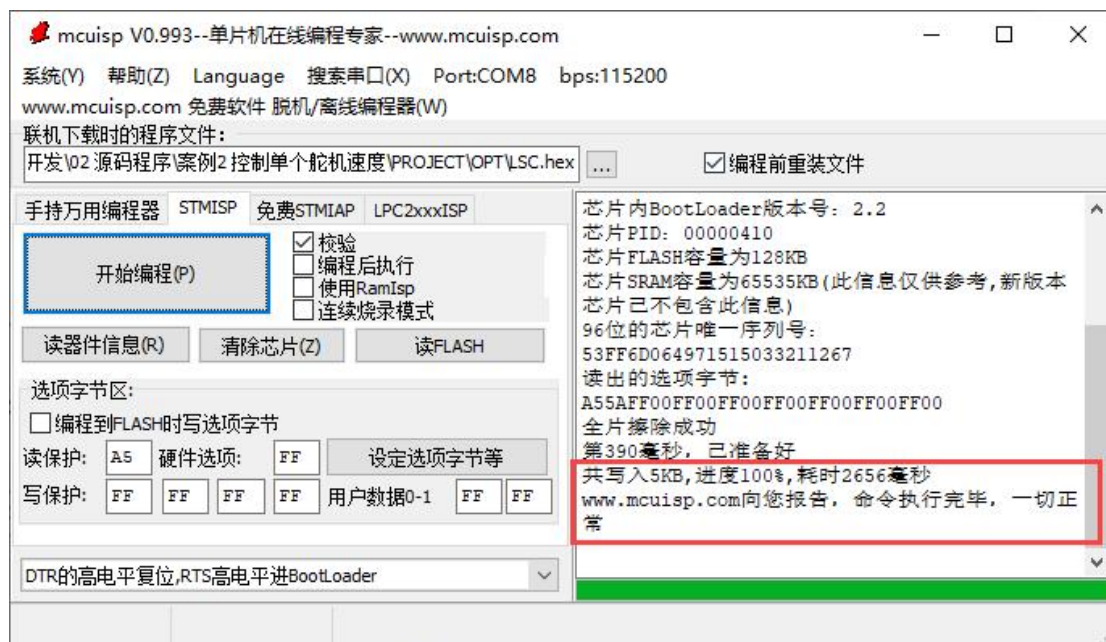


点击“”，在“总线舵机控制器二次开发教程\04 STM32 版本开发\02 源码程序\案例 2 控制单个舵机速度\PROJECT\OPT”文件夹下找到 hex 文件：





打开程序文件后，点击“开始编程”即可将程序下载到 STM32 单片机上（注意：在下载程序时，需要先将 STM32 单片机上的跳线帽拔掉并按下复位按键，当 mcuisp 右下角显示“命令执行完毕，一切正常”则说明下载完成，最后将跳线帽插回 STM32 单片机。）



下载完成后，打开总线舵机控制器电源开关，按下 STM32 单片机的复位按键，1 号舵机开始转动。

### 2.2.2 实现效果

程序运行后，舵机以 1200ms 的速度从 0 位置转动到 1000 位置，再以 500ms 速度转动从 1000 位置转动到 0 位置。

### 2.2.3 案例程序简要分析

- 导入必要功能包

在 main.c 程序中导入包：

```
#include "delay.h"
#include "uart.h"
#include "LobotServoController.h"
#include "bool.h"
```

功能包主要封装用于总线控制板通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机。

```
void delay_s(int s)
{
    for(;s>0;s--)
        delay_ms(1000);
}
```

在 main.c 文件中，有一个延时函数，该函数用于在单位为：秒（s）的延时。

- 串口初始化

```
void uartInit(u32 bound)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    USART_InitTypeDef USART_InitStructure;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1 | RCC_APB2Periph_GPIOA | RCC_APB2Periph_AFIO, ENABLE);

    //USART1_TX PA9
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART1_RX PA10
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART 配置
    USART_InitStructure.USART_BaudRate = bound; //波特率
    USART_InitStructure.USART_WordLength = USART_WordLength_8b; //数据位
    USART_InitStructure.USART_StopBits = USART_StopBits_1; //停止位
    USART_InitStructure.USART_Parity = USART_Parity_No; //校验位
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None; //流控制
    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //发送接收

    USART_Init(USART1, &USART_InitStructure);
    uartNVICInit();
    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
    USART_Cmd(USART1, ENABLE);
}
```

在 `uart.c` 文件下，初始化串口，配置 PA9 为 TX 输出数据，PA10 为 RX 读取数据。配置 USART 为 8 为数据位，1 位停止位，无校验位，无流控制，可读可写模式。

- 主程序

```
int main(void)
{
    SystemInit(); //系统时钟等初始化
    delay_init(72); //延时初始化
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //设置NVIC中断分组2:2位抢占优先级, 2位响应优先级
    uartInit(9600); //串口初始化为9600

    while(1) {
        moveServo(1, 1000, 1200); //1号舵机至1000位置
        delay_s(2);
        moveServo(1, 0, 1200); //1号舵机至0位置
        delay_s(2);
        moveServo(1, 1000, 500); //1号舵机至1000位置
        delay_s(1);
        moveServo(1, 0, 500); //1号舵机至0位置
        delay_s(1);
    }
}
```

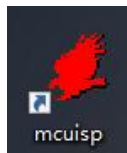
在 while 循环中, 通过调用 moveServo ( ) 函数, 控制 1 号舵机以 1200ms 的时间转动到位置 1000, 再转动到位置 0。再以同样的方式, 使舵机以 500ms 的时间转动。

## 2.3 案例 3 控制多个舵机转动

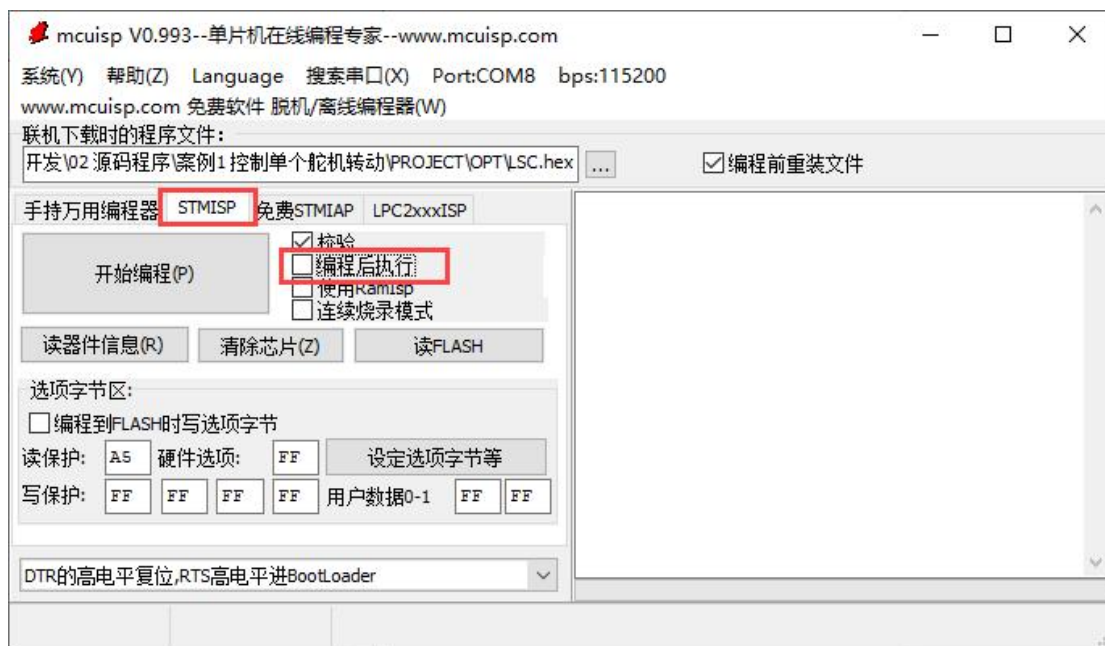
本案例串口向总线控制板发送指令, 再通过总线控制板控制多个舵机转动。

### 2.3.1 运行程序

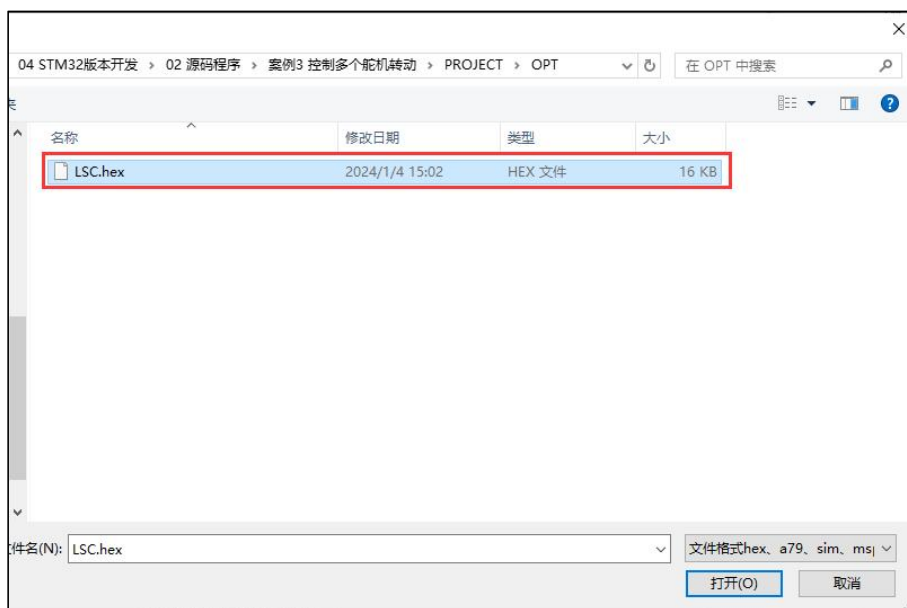
在“2.软件工具->STM32 软件工具”中打开 mcuisp 下载工具:



mcuisp 配置如下图:

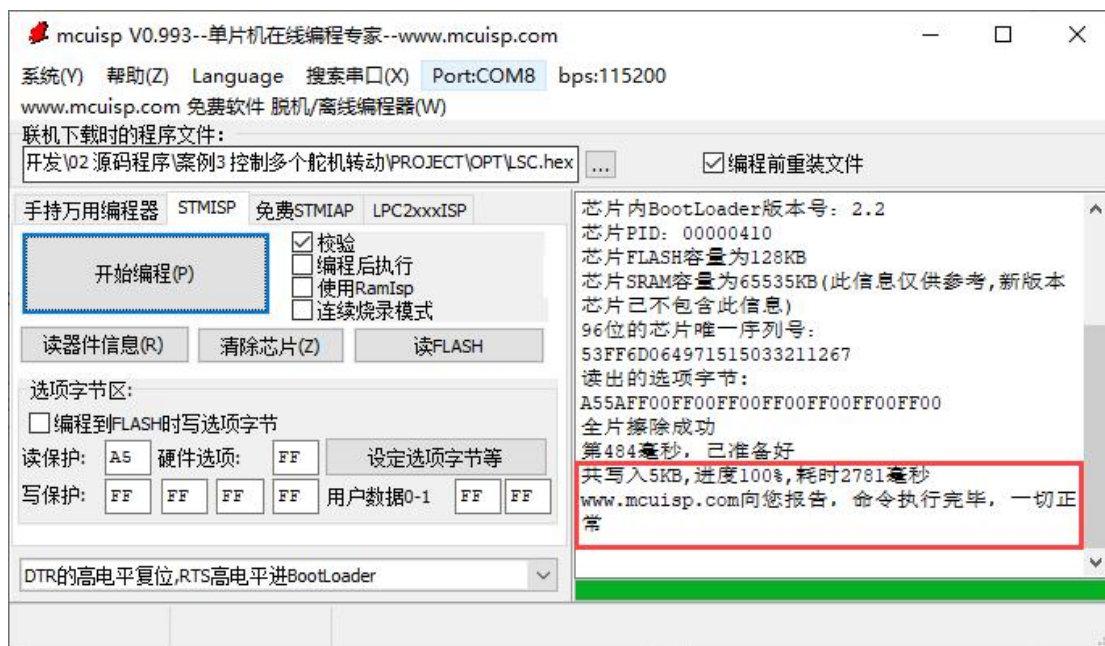


点击“”，在“总线舵机控制器二次开发教程\04 STM32 版本开发\02 源码程序\案例 3 控制多个舵机转动\PROJECT\OPT”文件夹下找到 hex 文件：



打开程序文件后，点击“开始编程”即可将程序下载到 STM32 单片机上（注意：在下载程序时，需要先将 STM32 单片机上的跳线帽拔掉并按下复位按键，当 mcuisp 右下角显示“命令执行完毕，一切正常”则说明下载完成，最后将跳线帽插回 STM32 单片机。）





下载完成后, 打开总线舵机控制器电源开关, 按下 STM32 单片机的复位按键, 1 号舵机开始转动。

### 2.3.2 实现效果

程序运行后, 舵机 1、舵机 2 和舵机 3 会从位置 0 转动到位置 1000, 以 1.5 秒为间隔来回转动。(可操控舵机数量有控制板的接口所定, 这里仅操控 3 个, 用户可通过代码尝试控制更多舵机。)

### 2.3.3 案例程序简要分析

- 导入必要功能包

在 main.c 程序中导入包:

```
#include "delay.h"
#include "uart.h"
#include "LobotServoController.h"
#include "bool.h"
```

功能包主要封装用于总线控制板通信的各功能模块, 我们可以使用其中定义的变量和函数来控制舵机。

```
void delay_s(int s)
{
    for(;s>0;s--)
        delay_ms(1000);
}
```

在 main.c 文件中，有一个延时函数，该函数用于在单位为：秒（s）的延时。

## ● 串口初始化

```
void uartInit(u32 bound)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    USART_InitTypeDef USART_InitStructure;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1 | RCC_APB2Periph_GPIOA | RCC_APB2Periph_AFIO, ENABLE);

    //USART1_TX PA9
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART1_RX PA10
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART 配置
    USART_InitStructure.USART_BaudRate = bound; //波特率
    USART_InitStructure.USART_WordLength = USART_WordLength_8b; //数据位
    USART_InitStructure.USART_StopBits = USART_StopBits_1; //停止位
    USART_InitStructure.USART_Parity = USART_Parity_No; //校验位
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None; //流控制
    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //发送接收

    USART_Init(USART1, &USART_InitStructure);
    uartNVICInit();
    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
    USART_Cmd(USART1, ENABLE);
}
```

在 uart.c 文件下，初始化串口，配置 PA9 为 TX 输出数据，PA10 为 RX 读取数据。配置 USART 为 8 为数据位，1 位停止位，无校验位，无流控制，可读可写模式。

## ● 定义舵机结构体

```
typedef struct _lobot_servo_ { //舵机ID, 舵机目标位置
    uint8_t ID;
    uint16_t Position;
} LobotServo;
```

将我们需要控制的舵机及信息存放在 LobotServo 的结构体中，主要用于多个舵机的控制。

- 控制舵机转动

```
LobotServo servos[3];

int main(void)
{
    SystemInit(); //系统时钟等初始化
    delay_init(72); //延时初始化
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //设置NVIC中断分组2:2位抢占优先级, 2位响应优先级
    uartInit(9600); //串口初始化为9600
    servos[0].ID = 1; //设置舵机ID
    servos[1].ID = 2;
    servos[2].ID = 3;

    while(1)
    {
        servos[0].Position = 0; //设置舵机位置为0
        servos[1].Position = 0;
        servos[2].Position = 0;

        moveServosByArray(servos, 3, 1200); //让3个舵机转动到指定位置, 运行时间为1200ms
        delay_s(2);

        servos[0].Position = 1000; //修改位置为1000
        servos[1].Position = 1000;
        servos[2].Position = 1000;

        moveServosByArray(servos, 3, 1200); //让3个舵机转动到1000的位置, 运行时间为1200ms
        delay_s(2);
    }
}
```

通过封装在结构体中的舵机信息, 通过串口发送给总线控制板, 控制板收到指令解析出我们要操控的舵机, 就会控制相对应的舵机转动到预设的位置。

## 2.4 案例 4 舵机中位及偏差调节

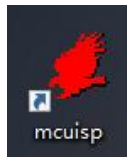
本案例串口向总线控制板发送指令, 通过总线控制板控制舵机转动中位及偏差位置。

**中位为假定即舵机初始的位置, 是以此位置为“零点”进行正负角度转动, 所以舵机在安装到舵盘前需调至中位再进行安装。**

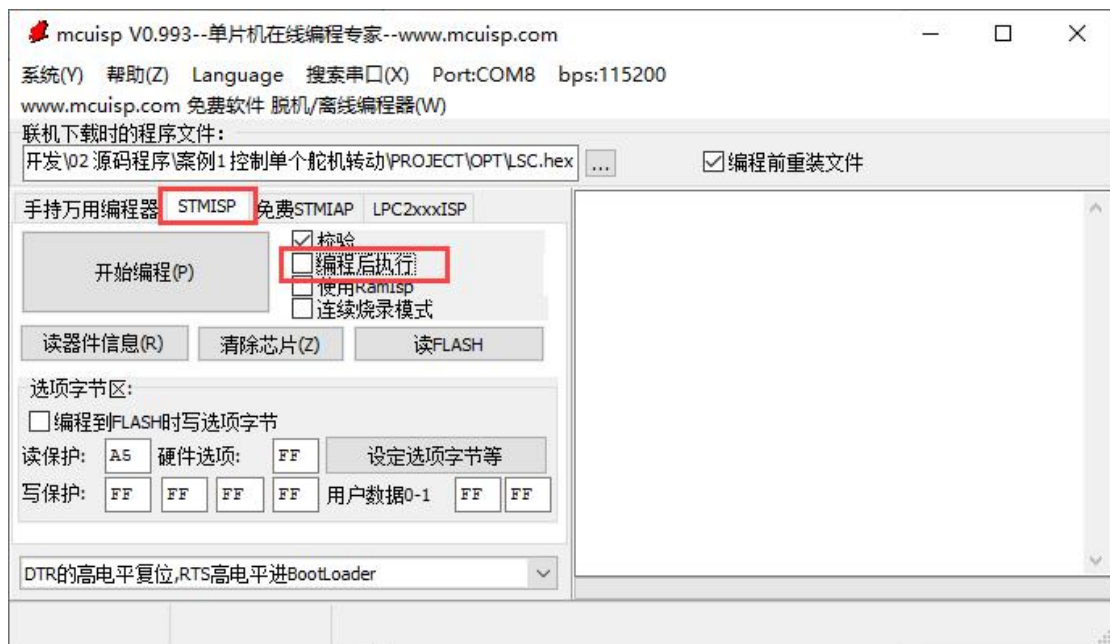
**舵机偏差是舵臂上花键的齿的间隔导致, 属于机械误差, 通过程序可以调节。在机器人或机械臂中, 如果不对偏差进行调节, 可能会导致机体运动受限, 从而影响一些运动效果。**

### 2.4.1 运行程序

在“2.软件工具->STM32 软件工具”中打开 mcuisp 下载工具:



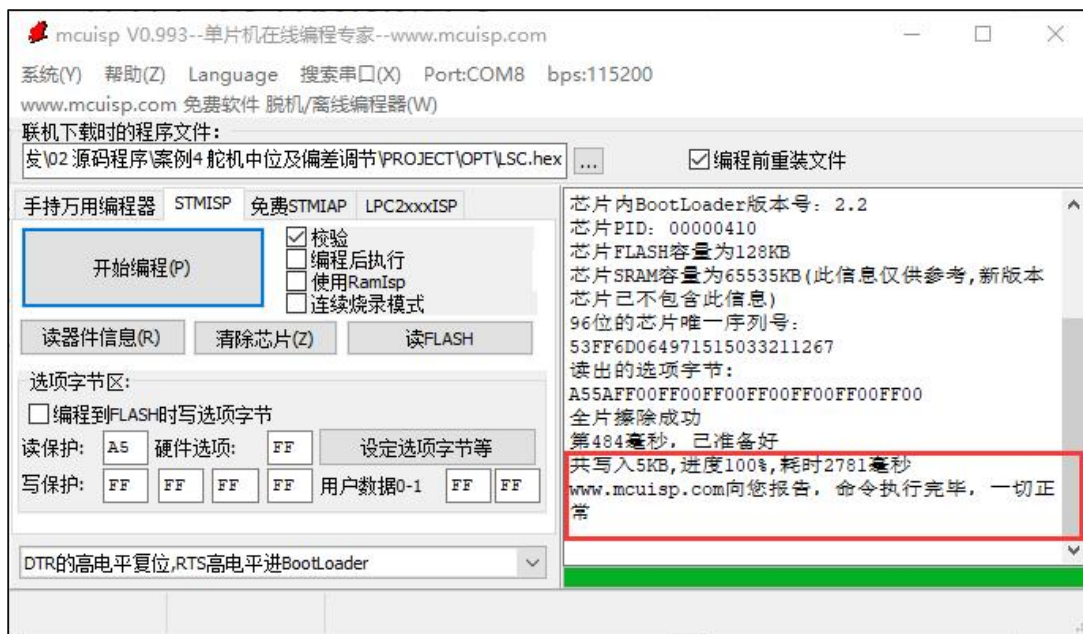
mcuisp 配置如下图：



点击“”，在“总线舵机控制器二次开发教程\04 STM32 版本开发\02 源码程序\案例4 舵机中位及偏差调节\PROJECT\OPT”文件夹下找到 hex 文件：



打开程序文件后，点击“开始编程”即可将程序下载到 STM32 单片机上（注意：在下载程序时，需要先将 STM32 单片机上的跳线帽拔掉并按下复位按键，当 mcuisp 右下角显示“命令执行完毕，一切正常”则说明下载完成，最后将跳线帽插回 STM32 单片机。）



下载完成后，打开总线舵机控制器电源开关，按下 STM32 单片机的复位按键，1 号舵机开始转动。

### 2.4.2 实现效果

舵机首先回到中位，在短暂的延时过后，转动到偏差角度并继续保持。

### 2.4.3 案例程序简要分析

- 导入必要功能包

在 main.c 程序中导入包：

```
#include "delay.h"
#include "uart.h"
#include "LobotServoController.h"
#include "bool.h"
```

功能包主要封装用于总线控制板通信的各功能模块，我们可以使用其中定义的变量和函



数来控制舵机。

```
void delay_s(int s)
{
    for(;s>0;s--)
        delay_ms(1000);
}
```

在 main.c 文件中，有一个延时函数，该函数用于在单位为：秒（s）的延时。

### ● 串口初始化

```
void uartInit(u32 bound)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    USART_InitTypeDef USART_InitStructure;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1 | RCC_APB2Periph_GPIOA | RCC_APB2Periph_AFIO, ENABLE);

    //USART1_TX PA9
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART1_RX PA10
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART 配置
    USART_InitStructure.USART_BaudRate = bound; //波特率
    USART_InitStructure.USART_WordLength = USART_WordLength_8b; //数据位
    USART_InitStructure.USART_StopBits = USART_StopBits_1; //停止位
    USART_InitStructure.USART_Parity = USART_Parity_No; //校验位
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None; //流控制
    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //发送接收

    USART_Init(USART1, &USART_InitStructure);
    uartNVICInit();
    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
    USART_Cmd(USART1, ENABLE);
}
```

在 uart.c 文件下，初始化串口，配置 PA9 为 TX 输出数据，PA10 为 RX 读取数据。配置 USART 为 8 为数据位，1 位停止位，无校验位，无流控制，可读可写模式。

### ● 控制舵机转动

```
int main(void)
{
    int deviation;
    SystemInit(); //系统时钟等初始化
    delay_init(72); //延时初始化
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //设置NVIC中断分组2:2位抢占优先级，2位响应优先级
    uartInit(9600); //串口初始化为9600

    moveServo(1, 500, 500); //1号舵机，转动到位置500，用时500ms
    delay_s(2); //延时程序，这里延时2s
    deviation = 80;
    moveServo(1, 500+deviation, 200); //1号舵机，转动到位置580，用时200ms
    while(1);
}
```

主程序运行前，定义舵机偏差，首先让舵机转动到中位，延时 2 秒后在中位的基础上再增加一个偏差，让舵机转动到这个偏差的位置，并一致保持在偏差的角度位置。